

1. AI Kullanım Alanları ve Mimari Yapı

Sistemde Gemini'nin görev yapacağı 4 ana istasyon bulunmaktadır:

- Akıllı Anlamsal Arama (Senaryo 1, 2, 3):** Kullanıcının kelimelerine değil, niyetine odaklanır. "Yurt fiyatı" ile "Barınma maliyeti" arasındaki bağı kurarak kullanıcıyı doğru senaryoya yönlendirir.
- İçerik Moderasyonu (Güvenlik Kapısı):** Soru gönderildiği anda hakaret, spam veya kişisel veri kontrolü yapar. Bu aşamada kategori ataması yapılmaz.
- Soru Normalleştirme (Editör Desteği):** Admin paneline düşen ham öğrenci sorularını resmi, net ve FAQ formatına uygun bir soru cümlesine dönüştürür.
- Kategori Önerisi:** Normalleştirilen sorunun hangi başlık (Burs, Sınav vb.) altına gireceğini tahmin eder.

2. Teknoloji ve Yöntemler

Bu alanları hayata geçirmek için gereken mühendislik araçları:

- Model:** Hız ve maliyet verimliliği için tüm noktalarda **Gemini-1.5-Flash** kullanılmalıdır.
- Vektörel Yakınlık (Semantic Search):** Cümleleri sayısal dizilere çevirmek için text-embedding-004 modeli ve bu verileri saklamak/sorgulamak için **pgvector (PostgreSQL)** veya **ChromaDB** gibi bir vektör veritabanı gereklidir.
- Debouncing (Frontend):** Arama motorunun gereksiz API çağrısı yapmaması için kullanıcının yazmayı bitirmesini bekleyen (500ms) mekanizma.
- Rate Limiting:** Kötü niyetli kullanımı engellemek için kullanıcı başına istek sınırı (Örn: Dakikada 5 soru gönderimi).

3. Arayüzü Güçlendiren Canlılık Gereklilikleri

Kullanıcının sistemle "iletişimde" olduğunu hissetmesi için gereken UX öğeleri:

- AI İşlem Durumları:** Kullanıcı soru gönderdiğinde "AI kontrol yapıyor..." şeklinde bir yükleme ikonu ve ardından "Kontrol tamamlandı, sorunuz sıraya alındı" Toast mesajı gösterilmelidir.
- Dinamik Statü Takibi:** Öğrenci panelinde sorunun durumu (Bekliyor, Admin Onayında, Cevaplandı) ikon ve renk değişimleriyle anlık yansıtılmalıdır.
- Arama Görselleştirmesi:** Arama sonuçları gelirken "Size en uygun sonuçları buldum" gibi güven veren başlıklar ve akıllı kart tasarımları kullanılmalıdır.
- Modal Spinner:** Admin modali açıldığında Gemini'nin metni düzenlediğini gösteren özel bir AI animasyonu/spinner kullanılmalıdır.

4. Kritik Prompt Hazırlığı

Modelden en doğru veriyi almak için backend'de kullanacağın detaylı promptlar:

A. Arama ve Senaryo Analizi Promptu

"Sen bir üniversite bilgi asistanısın. Kullanıcı sorgusunu: '\$query' mevcut SSS listesiyle anlamsal olarak kıyasla. Eğer %85 ve üzeri anlam eşleşmesi varsa (Senaryo 1), 'directAnswer' döndür. Eğer tam eşleşme yoksa ama bekleyen sorularla ilgiliyse (Senaryo 2), benzerlerin ID'lerini döndür. Hiçbir bağ yoksa 'null' (Senaryo 3) döndür. Yanıt SADECE JSON formatında ver."

B. Moderasyon Promptu (Adım 2 Revizeli)

"Aşağıdaki soruyu içerik politikalarına göre denetle: '\$question'. Hakaret, spam veya uygunsuzluk varsa 'isAppropriate: false' döndür. Kategori ataması YAPMA. Yanıt SADECE JSON formatında ver."

C. Admin Normalize Promptu (Adım 4 Revizeli)

"Üniversite editörü rolündesin. Ham metni: '\$rawText' resmi ve anlaşılır bir soru cümlesine dönüştür. Ayrıca bu soru için en uygun kategoriyi (Burs, Kayıt, Diğer vb.) öner. Taslak cevap üretme. Yanıt SADECE JSON formatında ver."

Kübra, bu dökümlerle sistemin hem teknik omurgasını hem de kullanıcıyı etkileyen o "akıllı asistan" ruhunu tamamlamış oldun. Uygulamanın en çok hangi aşamasında (Örn: Vektörel arama algoritması) zorlanabileceğini düşünüyorsun?

1. Temel Kütüphaneler ve Kurulum

Sistemin beyni Gemini API olduğu için öncelikle Google'ın resmi SDK'sını kurmalısın.

Backend (Node.js/Python)

- **Google Generative AI:** Gemini ile iletişim kurmak için.
- **Dotenv:** API anahtarını güvenli bir şekilde .env dosyasında saklamak için.
- **Express-Rate-Limit:** Kötü niyetli kullanımı engellemek (Dakikada 5 soru gibi) için.

Bash

Node.js projesi için:

```
npm install @google/generative-ai dotenv express-rate-limit [cite: 89, 134]
```

Python projesi için:

```
pip install google-generativeai python-dotenv [cite: 89]
```

Veritabanı (Vektörel Arama İçin)

Senaryo 1 ve 2'deki anlamsal benzerliği (yurt/barınma ilişkisi gibi) kurabilmek için PostgreSQL kullanıyorsan şu eklentiye kurmalısın:

- **pgvector:** Vektör verilerini saklamak ve mesafe ölçmek için kullanılır.

2. Proje Yapılandırması (Configuration)

API anahtarını asla doğrudan koda yazma; bu büyük bir güvenlik açığıdır. Tüm çağrılar backend üzerinden yapılmalıdır.

.env Dosyası:

Kod snippet'i

GEMINI_API_KEY=AlzaSy_Kendi_Anahtarini_Yaz [cite: 92]

gemini.js (İstemci Başlatma): Projede maliyet ve hız dengesi için **Gemini-1.5-Flash** modelini kullanacağız.

JavaScript

```
// src/config/gemini.js
```

```
const { GoogleGenerativeAI } = require('@google/generative-ai'); [cite: 94]
```

```
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY); [cite: 94]
```

```
// Moderasyon, Arama ve Normalize işlemleri için Flash modelini kullanıyoruz [cite: 94, 140]
```

```
const flashModel = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' }); [cite: 94]
```

```
module.exports = { flashModel }; [cite: 94]
```

3. Kritik İşlevler İçin Servis Yapıları

A. Moderasyon ve Senaryo Analizi

Öğrenci soru gönderdiğinde veya arama yaptığında çalışacak olan JSON tabanlı servis:

- **JSON Parse Güvenliği:** Gemini bazen yanıtı Markdown blokları içinde verebilir; bunu temizlemek için bir yardımcı fonksiyon (safeParseJSON) kullanmalısınız.
- **Fallback (Hata Yönetimi):** Gemini API hata verirse (kota aşımı vb.) sistemin çökmemesi için mutlaka try/catch blokları eklemelisiniz.

B. Canlılık Gereksinimleri (Frontend)

Kullanıcıya o "canlı" hissiyatı vermek için şu kütüphaneleri kullanabilirsiniz:

- **React-Hot-Toast / SweetAlert2:** AI kontrolü başladığında ve bittiğinde anlık bildirimler göstermek için.
- **Lodash (Debounce):** Kullanıcı arama çubuğuna yazarken her harfte API'yi yormamak, sadece yazma bitince (500ms) arama yapmak için.

4. Admin Paneli İçin Hazırlık

Admin "Cevapla" butonuna bastığında modal açılırken Gemini'den gelen normalize edilmiş veriyi (normalizedQuestion) ve kategori önerisini (suggestedCategory) göstermek için backend'de bir "Prepare" endpoint'i oluşturmalısın.

Kübra, kurulumları tamamladıktan sonra ilk iş olarak **.gitignore** dosyasına .env eklediğinden emin ol. Güvenlik, mühendisliğin ilk kuralıdır.